Canonical Peer Review

Lean on Vampire Models

Manuscript subtitle: *Kernel-Checked Finite Counter-Models for Lean 4: a Verified Reflective Checker, a Measured Scaling Recipe, and a Million-Fact Certificate*

Authors: Michael Jagdeo (Exponent Labs LLC) and Jeff Dumps

Reviewer: Manus AI **Review conducted:** 15–16 October 2026

Final reviewed release: `e840242664a91ecf861a82fd49d7488c042f1c76` (round 4c)

Reported benchmark evaluation: `66c8b32`

Recommendation: Accept as a scoped formal-methods tool-and-engineering contribution.

Review disclosure. This is an AI-generated technical peer review conducted by Manus at the authors' request through an iterative manuscript and artifact review. Independent execution below means reviewer-run commands and checks, distinct from the authors' supplied logs; it does not mean an independent human or institutional reviewer. This report is not an anonymous human review, a venue acceptance decision, or a universal correctness certification. It consolidates and supersedes the earlier provisional recommendations for the final reviewed release, while retaining the material findings and assurance limits.

Overall assessment

The paper presents a useful connection between external finite-model search and Lean 4 certification. Its strongest contribution is not a new model-finding algorithm, nor the invention of proof by reflection, but an implemented combination of Vampire model import, two complementary certificate representations, a reusable verified finite-model evaluator, and measured techniques for making large certificates practical.

The final artifact provides a substantially more persuasive correctness and reproducibility case than the initial manuscript. The review included primary-source comparison, inspection of the formal checker and trusted translation stages, reconstruction of supplied source packages, executed regression tests, fresh kernel replay, adversarial reproductions, complete-source recertification of affected examples, and a genuine rerun of the philosophical dependency map. Material defects were found, disclosed and repaired rather than explained away by unchanged benchmark totals.

I recommend acceptance within the stated scope, with no outstanding mandatory correction identified in the final release. This judgment rests on the distinction between kernel validity of the emitted statement and fidelity of that statement to its source. The former has a formal checker and executed proof-validation evidence; the latter remains a tested, trusted translation responsibility with explicitly bounded empirical coverage.

Contribution and relationship to prior work

The tool offers two certificate paths that address different computational costs. The shallow path represents the finite carrier as a Lean inductive type, interprets symbols directly and proves the translated formulas as propositions. The reflected path encodes formulas and model tables as data and applies a once-proved evaluator. This division is valuable: reducing proposition elaboration helps very large ground-fact workloads, while quantified formulas can retain a different evaluation bottleneck.

The novelty claim is appropriately an integration claim. Goubault-Larrecq's finite-model work for first-order security-protocol clauses is a close precedent for model checking with formally checkable Coq proof

output. Mechanized finite model theory in Coq supplies further semantic background. The Equational Theories Project is a substantial precedent for large-scale Lean-checked finite countermodels, but its central signature is the single binary operation of magmas. Nitpick provides broader higher-order model finding in Isabelle, with different scope and assurance conventions. *Lean on Vampire Proofs* is a complementary Vampire-to-Lean integration in the proof direction rather than the finite-model direction. [3–7]
Against that background, a reusable Vampire-to-Lean finite-model pipeline with the particular formal evaluator and engineering evidence supplied here is a worthwhile contribution. The literature review supports this scoped positioning, not an exhaustive claim that no related implementation exists or that first-order finite-model certification is unprecedented.
The negative engineering results also add value. The paper identifies workloads for which reflection regresses, optimization ideas that did not help, and translation errors that kernel acceptance alone could not detect. Such results make the scaling discussion more useful than a collection of favorable timings.

Technical correctness and trust boundaries

The reflected checker has an adequate semantic contract

An early review concern was that truth under a bounded evaluator need not imply ordinary first-order satisfaction. An empty bounded domain permits vacuous universal truth, and unrestricted natural-number interpretations can place constants or function results outside the quantified domain. These were specification-level objections to an insufficient contract, not demonstrations that Lean's kernel proved a false theorem.
The final checker addresses that gap. Its validity and well-formedness conditions enforce a nonempty domain, closure of the relevant function interpretations, in-range constants and properly scoped terms. The semantic bridge connects bounded evaluation to ordinary satisfaction in a structure on `Fin n`. The final reflected certificate retains both formula well-formedness and truth, rather than applying an evaluator-correctness theorem without its necessary hypotheses.
Source inspection found no confirmed soundness defect in this theorem chain. Executed builds and fresh replay of the checker and representative certificate libraries support that assessment. The shallow path likewise makes nonemptiness an explicit kernel-checked conjunct and rejects a zero-sized candidate before emission.
This is a meaningful formal guarantee about the declared formula data. It is not a theorem that arbitrary serialized bytes or arbitrary source files have been faithfully interpreted.

The translator is trusted, not formally verified

Record selection, source roles, symbol mapping, preprocessing, rewriting and source-to-Lean translation determine which mathematical statement is certified. Packed encodings and decoder metadata can describe a different formula list if those trusted stages are wrong. A proof about that different list may be a valid Lean proof while failing to establish the intended source claim. Lean's proof-validation guidance distinguishes proof validity from the meaning of its statement. [1]
The final paper makes this boundary explicit. Its independent source oracle has a separately written record reader and formula evaluator, although it still shares the formula lexer and variable collector. That is stronger than re-evaluating production-rewritten formulas, but it does not eliminate all correlated failure modes or constitute a verified compiler.

The appropriate interpretation is therefore **a verified finite-model checker behind a tested, trusted front end**. No broader end-to-end compiler-verification claim is needed for acceptance of the contribution reported here.

Material review findings and their resolution

The most consequential executed finding was a source-omission defect. The original reader searched for matching records instead of consuming the complete input. Legal quoted record names or comments between tokens could cause an axiom to disappear silently. The test oracle had copied the same record-selection pattern. With contradictory two-record inputs and a deliberately erroneous candidate satisfying only the surviving record, both certificate paths accepted proofs of the weakened translated problem.

This did not refute Lean's kernel or the final checker theorem. It demonstrated a failure at exactly the trusted source-fidelity boundary. Eight earlier benchmark certificates omitted premises and were withdrawn. The repaired pipeline then certified the complete supplied problems, and I independently regenerated and compiled all eight complete-source cases. **Unchanged strict totals do not validate the superseded certificates; the complete-source certifications are new results.**

The final reader consumes the file structurally or rejects it, and the oracle reads records independently. The original contradiction fixtures now retain both records and reject the partial-model candidate. Related repairs also address generated-name collisions, malformed delimiter kinds, empty fields and variable capture.

Two ancillary defects were independently reproduced and corrected. The ablation driver originally collapsed distinct legal names into shared experiment identifiers and silently dropped accepted roles outside its intended scope. It now uses collision-free internal identities and rejects unsupported ablation roles before producing a report. The collision fixture produces six distinct experiments, and the original role-loss fixture is rejected rather than analyzed as a reduced theory.

Finder-error accounting was also tightened. Explicit SZS errors, arbitrary internal-error termination text and missing executables now produce finder errors instead of ordinary unsuccessful searches, while documented resource timeouts remain inconclusive and uncached. The final cache repair applies the same policy to cache hits: only permitted cacheable answers are used; older error-bearing entries are ignored and the finder is run again. The original reviewer diagnostic, run unmodified, confirms that this last concern is resolved.

These repairs were accompanied by retained regressions and corrected documentation. The review also prompted explicit input-reconstruction checks and disclosure of an earlier include-expansion defect affecting commented directives. The complete benchmark reconstruction remains supplied evidence rather than an independently repeated full-corpus operation.

What the reviewer independently verified

The review was not confined to manuscript claims. Source packages and subsequent deltas were checked against manifest byte counts and SHA-256 hashes and applied in separate review trees. The final ten-file delta and its previous-version checks succeeded. Commit identifiers are those declared in the manifests; the inspected file chain was hash-verified rather than independently retrieved as a complete Git checkout.

Using Lean 4.35.0 and Vampire 5.1.0, the reviewer independently established the following:

- **Final build and suite:** the round-4c project builds, and the full supplied suite completes with **168 passed and two skipped**, exit status zero. The skips require the omitted full TPTP corpus and benchmark Vampire cache. The authors' separate full-data log reports 170 passing tests; that is not the reviewer's executed count.
- **Fresh proof validation:** the round-4b `recheck.sh` run succeeds for `Checker`, `AllShallow` and `AllReflected`, including the 30 shipped example certificates and their axiom audits. In round 4c, the checker, both aggregates, those 30 certificates and replay script—34 files—are byte-identical. The unchanged replay was not repeated solely for the cache repair.
- **Adversarial source and model controls:** the original source-omission protocol, wrong-model rejection tests, naming-collision controls, empty-domain and malformed-data boundary cases are exercised through the supplied suite and targeted reproductions.
- **Affected complete problems:** the eight SWW cases—`SWW865+1`, `SWW882+1`, `SWW883+1`, `SWW889+1`, `SWW902+1`, `SWW921+1`, `SWW938+1` and `SWW942+1`—are independently regenerated and compiled on their complete supplied expanded inputs.
- **Source-preservation comparison:** old and repaired readers and problem transforms agree on 24 supplied small inputs: 15 examples and nine complete expanded benchmark inputs. The broader 316-input comparison is supplied evidence, not a reviewer rerun of all those inputs.
- **Philosophical integration:** the supplied `AutoCM.A12` and `AutoCM.A15` bridges compile against Nakamura's exact cited upstream revision using Lean 4.13; their result axiom audits report `propext` only. The repaired ablation driver reproduces all 36 register verdicts/model sizes and both greedy sufficient sets in a genuine Vampire/Lean execution.
- **Cache repair:** the unmodified diagnostic rejects both fresh and cached internal-error responses. A genuine valid cache entry is reused without rewriting it, and both invocations emit the committed certificate.

These independently executed subsets should not be conflated with the authors' full benchmark replay or performance campaign. I did not regenerate all 185 strict benchmark certificates, replay the entire benchmark population, rebuild the omitted TPTP corpus, inspect the complete omitted cache inventory, or repeat the million-fact timing measurements.

Evaluation: useful descriptive evidence, not an untouched generalization test

The final reported evaluation covers three disjoint samples of 100 problems from the eligible untyped TPTP Satisfiable/CounterSatisfiable frame. These statuses do not imply finite satisfiability; the SZS ontology distinguishes the finite variants. Failure of finite-model search within its budget is not a proof that no finite model exists. [2]

The supplied final ledgers report:

- **Development sample:** 78 models found; 71 strict certificates, or 91.0% of found models and 71% of all sampled problems; two additional native-assisted results.
- **Second sample:** 62 models found; 55 strict certificates, or 88.7% of found models and 55% of all sampled problems; two additional native-assisted results.
- **Third sample:** 67 models found; 59 strict certificates, or 88.1% of found models and 59% of all sampled problems; three additional native-assisted results.

Thus the strict total is **185**, with **seven native-assisted outcomes** kept separate. The supplied replay records cover all 185 strict certificates, including the million-fact case. The review reconciled their memberships and outcomes but did not independently replay that complete population. Fresh replay uses Lean's own kernel; it is not validation by an independently implemented second kernel.

The development set is not uniformly sampled overall. The later samples were initially drawn uniformly, but subsequent debugging and repairs exposed all three sets to development. Their corrected results are valuable descriptive validation of the reported implementation, not an untouched estimate of out-of-sample generalization. Similar conditional rates do not establish equality or a causal explanation for differences in full-pipeline success. Adaptive reuse of evaluation data warrants precisely this qualification. [8]

The source-fidelity ledger reports successful original-formula evaluation for **189 of 192** certified outcomes: 184 of 185 strict and five of seven native-assisted results. `NLP219-1`, `NLP179+1` and `NLP186+1` remain unchecked after evaluation time limits. Record agreement or satisfaction of rewritten formulas does not close those original-formula gaps. The final documentation correctly keeps such checks separate.

Rewrite validation is likewise bounded. The full suite passes 18 adversarial formula fixtures over every interpretation in the stated 1,032-structure test space, and a deliberately wrong rewrite yields the retained 156-disagreement negative control. The supplied benchmark rewrite ledger reports 192 completed and 14 unchecked outcomes among 206 selected problems. These are valuable regression and differential checks, not a general proof of rewrite equivalence.

The retained `ALG310-1` native-assisted timeout is appropriately reported as a failure. Background contention in the corrected sample rerun is disclosed as a plausible contributor, not an established cause. The recorded translator crashes and remaining resource limitations are counted as failures, not removed from the evaluation to improve its headline rates.

Performance and case-study interpretation

The million-fact workload is a useful certificate-throughput stress test. The supplied matched comparison reports median wall times of **5.2 minutes reflected versus 14.1 minutes shallow**, approximately 2.7-fold, and a roughly 2.6-fold CPU-time improvement. Largest-process memory and aggregate sampled memory are distinguished rather than reported as a single interchangeable peak.

These measurements use matched Lean versions and worker budgets with alternating clean runs, three repetitions per method. Model search and the prebuilt checker are excluded, fresh replay is separate, and residual background activity is disclosed. The timing version is earlier than the evaluated/released code: the reflected million-fact modules are reported byte-identical, whereas the shallow modules require the disclosed helper renaming and `Nonempty T` change and were not re-timed. I inspected the supplied evidence and qualifications, but did not repeat those large timings.

The one-element model makes individual facts easy to evaluate. The result supports an engineering claim about representation, elaboration and build organization, not a general complexity claim for larger domains or deeply quantified theories. The paper's quantified-reflection regressions and pragmatic, incompletely validated dispatch heuristic remain important limits.

The Spinoza case study is useful as a formalization-integration demonstration. Generated two-element models are checked against the pinned upstream Lean classes rather than only a detached transcription. The dependency map concerns Nakamura's reconstructed first-order register, including fixed definitions

and auxiliary commitments—not an unqualified resolution of Spinoza's intended metaphysics. The register's nontrivial fields should not be mistaken for eight original stated axioms.

Negative dependency results are supported by finite countermodels. Positive ablation results rely on Vampire except where separately supported by the cited upstream Lean proofs. Leave-one-out necessity and reduced-set sufficiency are distinct; the greedy sets are not a claim of a unique minimum-cardinality basis. A countermodel establishes non-derivability in the specified theory, not automatically full independence. Smaller model cardinality is an automation result, not evidence of greater philosophical fidelity. [9]

Final recommendation and publication scope

The final work has an adequate correctness case for its decoded finite-model checker, substantial executed artifact evidence, an appropriately qualified performance result and a useful real-formalization case study. The authors' disclosure and repair of defects is a strength, particularly because the review demonstrated why kernel acceptance cannot replace source-fidelity checking.

I recommend acceptance of the final reviewed release as a formal-methods tool-and-engineering contribution. No mandatory correction remains from this review. A verified source translator, stronger serialized-format validation, fresh untouched evaluation samples and a broader multidimensional performance study would be valuable future work, but are not prerequisites for the scoped claims accepted here.

The recommendation applies to the identified artifact and evidence. It should not be quoted as a guarantee of universal correctness, unrestricted TPTP support, complete finite-model search, or independent reproduction of every published benchmark. For a durable publication record, this review should accompany the exact reviewed release and the retained reviewer execution records.

Sources consulted during the review

The manuscript and supplied artifacts are the sources for implementation measurements and benchmark outcomes. The primary references below were used for semantic, validation, novelty and evaluation context; they do not independently establish this tool's experimental results.

[1] Lean 4.35 Language Reference, *Validating a Lean Proof*, and the official Lean checker repository.

[2] TPTP, *Language Documentation*, and the SZS semantic success ontology.

[3] Jean Goubault-Larrecq, *Finite Models for Formal Security Proofs*, *Journal of Computer Security*, 2010.

[4] Dominik Kirst and Dominique Larchey-Wendling, *Trakhtenbrot's Theorem in Coq: A Constructive Approach to Finite Model Theory*.

[5] *The Equational Theories Project: Advancing Collaborative Mathematical Research at Scale*, arXiv:2512.07087.

[6] Jasmin Christian Blanchette and Tobias Nipkow, *Nitpick: A Counterexample Generator for Higher-Order Logic Based on a Relational Model Finder*.

[7] Bodingbauer et al., *Lean on Vampire Proofs (Short Paper)*, ITP 2026, DOI: 10.4230/LIPIcs.ITP.2026.36.

[8] Cynthia Dwork et al., *Generalization in Adaptive Data Analysis and Holdout Reuse*, NeurIPS 2015.

[9] Nakamura's *spinoza-ethica-lean* formalization, inspected at revision `6d4c0461265e50a3bbedb640fe60d9ad292b50a7`, particularly the definitions, axiom register, modal-form proofs and countermodels.